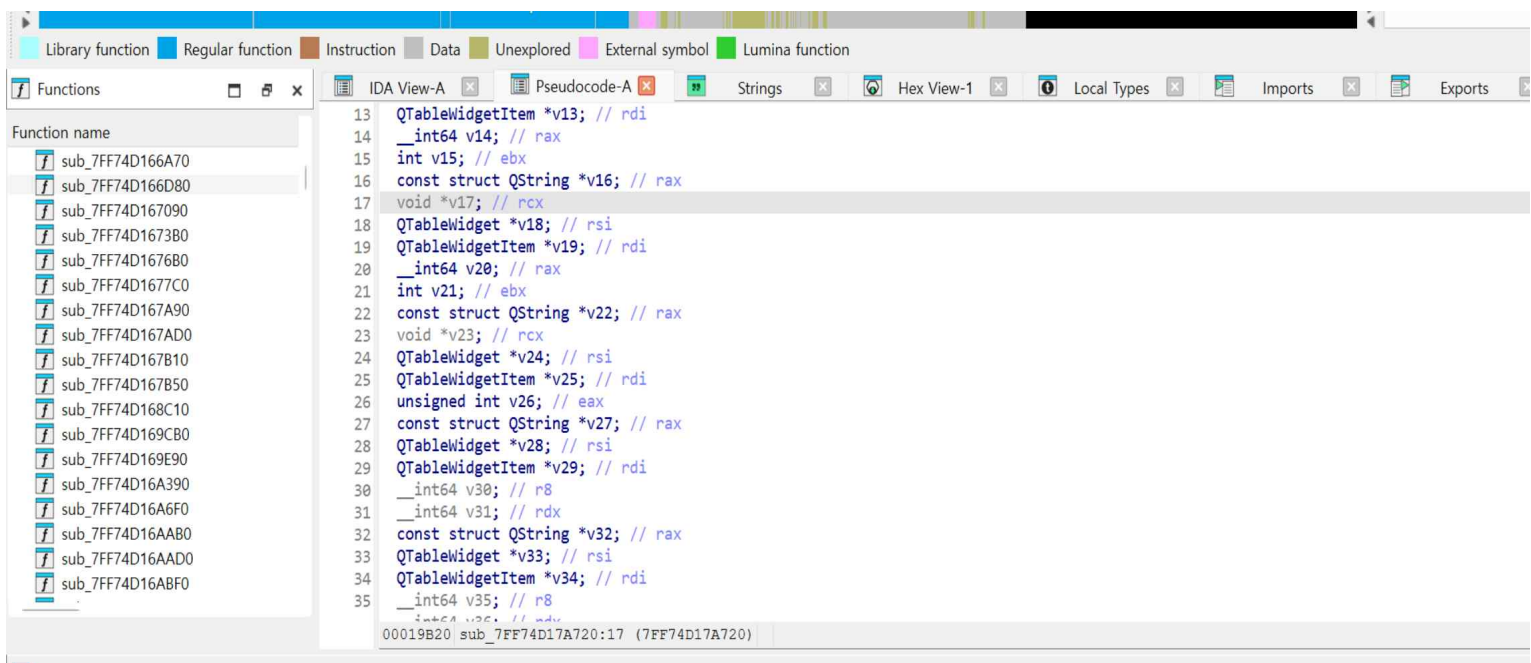


주식고수 공격을 위한 과정들

정명훈

처음으로 주식고수를 공격하기 위해서는 이 공격의 목적이 무엇인지부터 확정해야 한다. 나의 목적은 현재 가지고 있는 총 자산을 나는 10억 남들은 0원으로 보이게 출력하는 것이 목적이다.

이러한 목적을 달성하기 위해서는 우선 먼저 주식고수 실행파일을 한 번 실행하고 그 후에 IDA에 정적분석을 위해 EXE파일을 올려 보았다.



우선 IDA를 통해 보면 대부분의 함수들의 이름이 암호화가 되어있어 리버싱이 쉽지 않겠다는 것이 느껴진다. 하지만 당황하지 말고 이럴 때에는 먼저 SHIFT + F12를 눌러 문자열 부분을 한 번 확인해 보면

.rdata:00007FF...	00000000	C	ticket
.rdata:00007FF...	00000006	C	price
.rdata:00007FF...	0000000B	C	change_pct
.rdata:00007FF...	00000006	C	stock
.rdata:00007FF...	0000000A	C	avg_price
.rdata:00007FF...	0000000B	C	profit_pct
.rdata:00007FF...	00000006	C	total
.rdata:00007FF...	00000033	C	cannot use operator[] with a string argument with
.rdata:00007FF...	0000000B	C	MainWindow

이런 식으로 문자열까지는 암호화가 되어 있지 않아서 이 문자열들을 참조하는 위치에 가면 내가 원하는 목표를 이룰 수 있을 거 같으니 한 번 거기로 가보자

```

    j_j_free(v21);
}
}
v22 = *(QTableWidget **)(a1 + 104);
v23 = (QTableWidgetItem *)sub_7FF6C965C970(0x40u);
v31[3] = v23;
if ( v23 )
{
    v28 = 0;
    sub_7FF6C9658900(v12, "total", (__int64)&v28);
    LOBYTE(v24) = 102;
    v26 = (const struct QString *)QString::number(Src, v25, v24, 0);
    v2 |= 8u;
    LODWORD(v28) = v2;
    QTableWidgetItem::QTableWidgetItem(v23, v26, 0);
    *(_QWORD *)v23 = &off_7FF6C9663F60;
}

```

0001A65F sub_7FF6C965AE70:176 (7FF6C965B25F)

문자열 있는 곳 중 한 군데로 가보니 뭔가 이 부분은

주식고수			
TOMAS 님 보유 현금: 10000원			
주식 시세 포트폴리오 랭킹			
	순위	유저명	총자산
1	1	정명훈2	86675
2	2	asdf	17582
3	3	Social0w0	16735
4	4	tomas_3	10065
5	5	tomas6	10032
6	6	social0w0	10000
7	7	ababab	10000
8	8	qwer	10000
9	9	정명훈	10000
10	10	wjdaud	10000
11	11	tomas4	10000
12	12	TOMAS	10000

내 현재 포트폴리오를 보여주는 부분이라는 걸 쉽게 유추할 수 있다. 그렇다면 여기서

내가 관심있는 부분은 현재 총 자산인데 이는 total부분과 관련되어 있다는 걸

위의 스크린샷을 통해 쉽게 알 수 있다 이제 내 관심사는 이런 부분들에 대해서 하나씩 전부 다 어떤 값인지 추측하고 그 값을 저장하는 연산에 대해서 동적 디버깅을 통해 내 예측이 맞는지를 확인하는 것이다. 사람들이 가지고 있는 총 자산 관련된 변수는

```
v51[5] = v25;
if ( v23 )
{
    v28 = 0;
    sub_7FF6C9658900(v12, "total", (__int64)&v28);
    LOBYTE(v24) = 102;
    v26 = (const struct QString *)QString::number(Src, v25, v24, 0);
    v2 |= 8u;
    LODWORD(v28) = v2;
    QTableWidgetItem::TableWidgetItem(v23, v26, 0);
}
```

이 부분에 저장되어 있다는 걸 알아내고 sub_7FF6C9658900내부에 들어가 분석을 해보면

```
__int64 v3; // [prop_0001] [prop_0001] string
v3 = a2;
if ( *a1 != 1 )
{
    sub_7FF6C9656390(a1, a2, a3);
    v21 = sub_7FF6C9644840(Src, "cannot use value() with ");
    sub_7FF6C96454C0(&pExceptionObject, 306, v21, a1);
    CxxThrowException(&pExceptionObject, (_ThrowInfo *)&stru_7FF6C9669990);
}
pExceptionObject = a1;
v25 = nullptr;
v26[0] = 0;
v5 = 0x8000000000000000uLL;
v6 = 0x8000000000000000uLL;

case 2u:
LABEL_4:
    v26[0] = *(_QWORD *)*((__QWORD *)a1 + 1) + 8LL;
    break;
default:
    v26[1] = 0x8000000000000000uLL;
    switch ( v7 )
    {
        case 1LL:
            goto LABEL_6;
        case 2LL:
            goto LABEL_4;
        default:
            v6 = 1;
            v26[1] = 1;
            break;
    }
    break;
}
LABEL_6:
v8 = *(( int64 ***)a1 + 1);
```

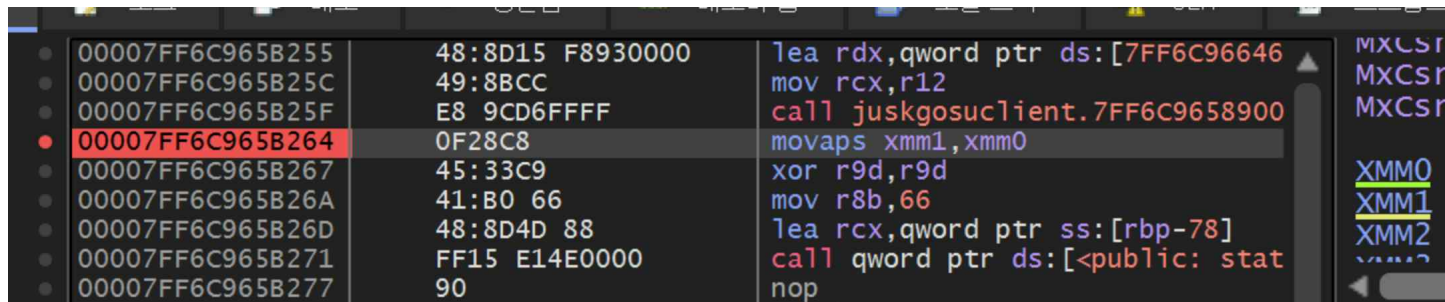
대략적으로 이 함수가 어떤 타입 검증을 거쳐 json에서 어떤 값을 가져오는 지를 알 수 있었다. (이 부분은 어셈블리와 디컴파일된 코드를 비교하면서 약간의 ai의 도움을 받아서 진행했다.) 그 다음에 이렇게 얻은 값을 보통 double type이면 xmm1에 보통 저장하는데 저장하는 부분이 디컴파일된 코드에는 보이지가 않아서 직접 어셈블리 부분을 파악하니

```

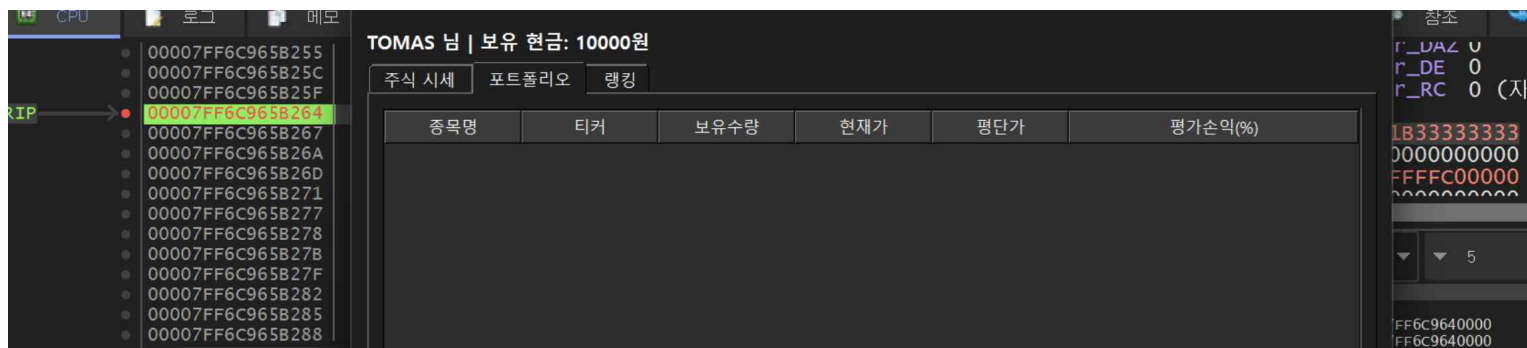
.text:00007FF6C965B25C      mov     rcx, r12
.text:00007FF6C965B25F      call    sub_7FF6C9658900
.text:00007FF6C965B264      movaps  xmm1, xmm0
.text:00007FF6C965B267      xor     r9d, r9d
.text:00007FF6C965B26A      mov     r8b, 66h ; 'f'
.text:00007FF6C965B26D      lea     rcx, [rbp+40h+Src]
.text:00007FF6C965B271      call    cs:?number@QString@@SA?AV1@NDH@Z ; QString::numb
.text:00007FF6C965B277      nop

```

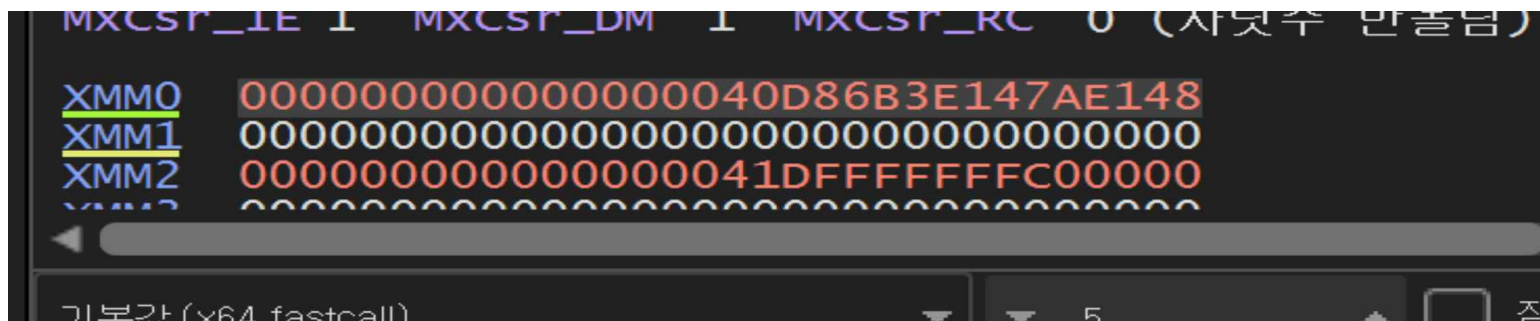
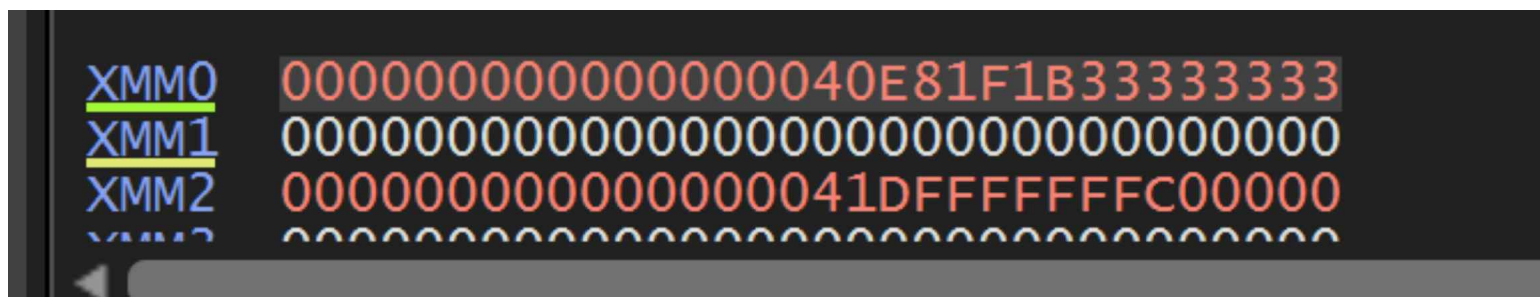
이런 식으로 xmm1에 현재 총 자산을 저장한다는 걸 확인할 수 있다 이제 내 예측이 맞는지 확인하기 위해서 xdbg를 사용해서 movaps xmm1, xmm0부분에 중단점을 설정하고

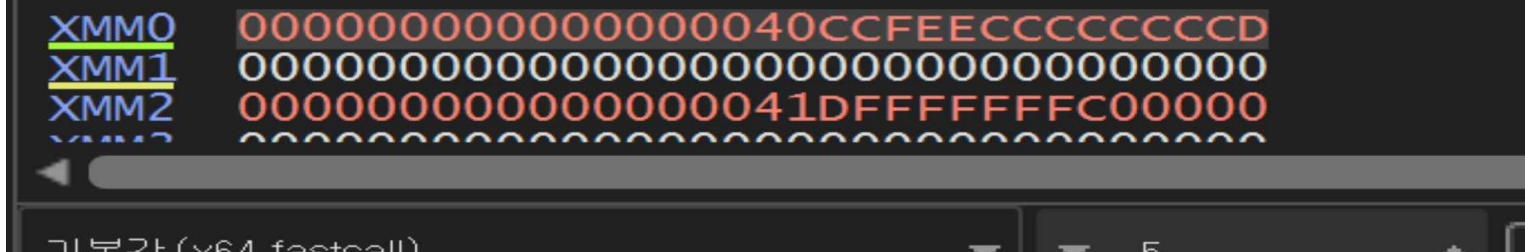


그 다음에 게임에서 랭킹창으로 이동하면 이렇게 중단점이 걸린다.

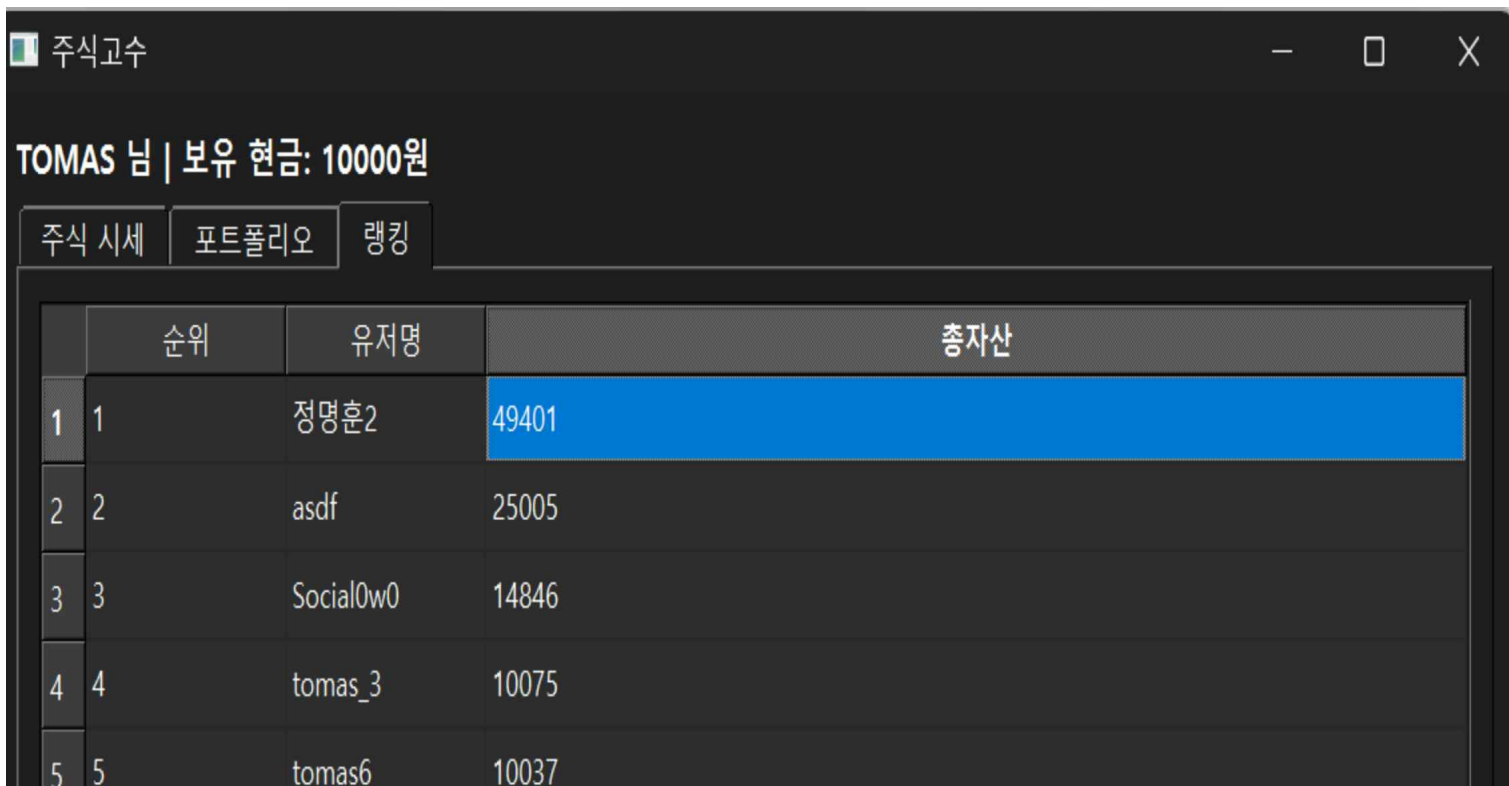


이제 xmm1값을 하나씩 확인하면





이렇게 하나씩 유저별로 총자산 금액을 하나씩 xmm0에 가지고 있다는 걸 알 수 있다.(중단점 기준 xmm1에 옮기기 이전이라 xmm1값은 0이다) 이게 맞다는 걸 확인하기 위해 10진수로 변환해 보면 각각 49392.85, 25004.97, 14845.85으로 중단점을 해제하고 랭킹에 가서 총 자산을 확인해보면



실제로 랭킹 1,2,3등의 자산이 xmm0에 저장된 값을 소수점 첫 번째 자리에서 반올림한 것이라는 것을 알았다 이제 그럼 내 총자산을 늘려보는 건 쉬운 일이다

TOMAS 님 | 보유 현금: 10000원

주식 시세

포트폴리오

랭킹

	순위	유저명	총자산
1	1	정명훈2	49401
2	2	asdf	25005
3	3	Social0w0	14846
4	4	tomas_3	10075
5	5	tomas6	10037
6	6	social0w0	10000
7	7	ababab	10000
8	8	qwer	10000
9	9	정명훈	10000
10	10	wjdaud	10000
11	11	tomas4	10000
12	12	TOMAS	10000

이제 총자산을 늘리는 건 디버거에서 현재 중단점에서 f9를 12번 눌러 내 순서가 오면 거기서 xmm0에 내가 원하는 금액을 적으면 된다.

TOMAS 님 | 보유 현금: 10000원

주식 시세

포트폴리오

랭킹

	순위	유저명	총자산
1	1	정명훈2	65452
2	2	asdf	21693
3	3	Social0w0	20312
4	4	tomas_3	10125
5	5	tomas6	10062
6	6	social0w0	10000
7	7	ababab	10000
8	8	qwer	10000
9	9	정명훈	10000
10	10	wjdaud	10000
11	11	tomas4	10000
12	12	TOMAS	1000000000

그럼 이렇게 내 총자산이 10억이 된다. 난 부자가 된 것이다. 다만 이는 다시 중단점을

해제하면 원래대로 돌아간다는 문제점이 있다.

이를 해결하기 위해서는 영구적인 패치가 필요한데 코드 케이브를 빈 공간에 작성한 뒤 거기에서 내가 원하는 올바른 함수를 호출하게 한 뒤에 내가 원하는 총액을 넣으려고 한다.

우선 코드 케이브를 text영역의 실행권한이 있는 빈 패딩부분을 찾아 작성하려고 하는데

•	00007FF6C965FEFD	0000	add byte ptr ds:[rax],al
•	00007FF6C965FEFF	0000	add byte ptr ds:[rax],al
•	00007FF6C965FF01	0000	add byte ptr ds:[rax],al
•	00007FF6C965FF03	0000	add byte ptr ds:[rax],al
•	00007FF6C965FF05	0000	add byte ptr ds:[rax],al
•	00007FF6C965FF07	0000	add byte ptr ds:[rax],al
•	00007FF6C965FF09	0000	add byte ptr ds:[rax],al
•	00007FF6C965FF0B	0000	add byte ptr ds:[rax],al
•	00007FF6C965FF0D	0000	add byte ptr ds:[rax],al
•	00007FF6C965FF0F	0000	add byte ptr ds:[rax],al
•	00007FF6C965FF11	0000	add byte ptr ds:[rax],al
•	00007FF6C965FF13	0000	add byte ptr ds:[rax],al
•	00007FF6C965FF15	0000	add byte ptr ds:[rax],al

스크린샷의 00007FF6C965FF01주소 부분 정도에 작성하는게 안전할 거 같아서 이 부분에

```
call 00007FF6C9658900
```

```
mov rax,41CDCD6500000000
```

```
movq xmm0,rax
```

```
ret
```

와 같이 올바른 원래의 함수 위치로 가서 함수를 실행한 뒤에 내가 원하는 금액 10억을 내 자산에 넣는 함수를 작성하면 된다.

•	00007FF6C965FEFF	0000	add byte ptr ds:[rax],al
•	00007FF6C965FF01	E8 FA89FFFF	call juskgosuclient.7FF6C9658900
•	00007FF6C965FF06	48:B8 0000000065CDCD	mov rax,41CDCD6500000000
•	00007FF6C965FF10	6648:0F6EC0	movq xmm0,rax
•	00007FF6C965FF15	C3	ret
•	00007FF6C965FF16	90	nop
•	00007FF6C965FF17	0000	add byte ptr ds:[rax],al
•	00007FF6C965FF19	0000	add byte ptr ds:[rax],al
•	00007FF6C965FF1B	0000	add byte ptr ds:[rax],al
•	00007FF6C965FF1D	0000	add byte ptr ds:[rax],al
•	00007FF6C965FF1F	0000	add byte ptr ds:[rax],al

• 00007FF6C965B25C	49:8BCC	mov rcx,r12
• 00007FF6C965B25F	E8 9D4C0000	call juskgosuclient.7FF6C965FF01
• 00007FF6C965B264	0F28C8	movaps xmm1,xmm0
• 00007FF6C965B267	45:33C9	xor r9d,r9d
• 00007FF6C965B26A	41:B0 66	mov r8b,66
• 00007FF6C965B26D	48:8D4D 88	lea rcx,qword ptr ss:[rbp-78]
• 00007FF6C965B271	FF15 E14E0000	call qword ptr ds:[<public: static class QString
• 00007FF6C965B277	90	nop
• 00007FF6C965B278	83CB 08	or ebx,8
• 00007FF6C965B27B	895C24 30	mov dword ptr ss:[rsp+30],ebx
• 00007FF6C965B27F	45:33C0	xor r8d,r8d
• 00007FF6C965B282	48:8BDB	mov rdx,rbp

그 다음 기존 함수 호출을 кей브 코드 주소로 이동하게 하면 된다.

주식고수

TOMAS 님 | 보유 현금: 10000원

주식 시세

포트폴리오

랭킹

	순위	유저명	총자산
1	1	정명훈2	1000000000
2	2	Social0w0	1000000000
3	3	tomas_3	1000000000
4	4	tomas6	1000000000
5	5	social0w0	1000000000
6	6	ababab	1000000000
7	7	qwer	1000000000
8	8	정명훈	1000000000
9	9	wjdaud	1000000000
10	10	tomas4	1000000000
11	11	TOMAS	1000000000
12	12	asdf	1000000000

하지만 이렇게 하고 실행하면 문제가 나의 총자산 뿐만 아니라 나머지 사람들의 자산 또한 전부 10억이 된다는 것이다. 이 문제를 해결하기 위해서 나는 우선 객체의 이름을 확인하고 이름이 나일때만 금액을 10억으로 바꾸게 해야 한다, 또 다른 문제점은 이렇게 패치를 해도 재실행 때마다 코드 кей브가 사라져서 코드 кей브의 위치를 변경하기로 했다.

그러기 위해서 우선 코드 кей브 위치를 찾아야 하는데 text영역에 빈 공간이 많지 않

고 뒤의 패딩 부분은 실제 텍스트 영역 밖의 공간이라 패치를 해도 다시 0으로 채워지는 문제가 생겼다.

• 00007FF7AA831782	CC	int3
• 00007FF7AA831783	E8 D79A0100	call juskgosuc1ient_v4.7FF7AA84B25F
• 00007FF7AA831788	48:B8 0000000065CDCD	mov rax,41CDCD6500000000
• 00007FF7AA831792	6648:0F6EC0	movq xmm0,rax
• 00007FF7AA831797	C3	ret
• 00007FF7AA831798	90	nop
• 00007FF7AA831799	90	nop
• 00007FF7AA83179A	57	push rdi
• 00007FF7AA83179B	48:83EC 40	sub rsp,40
• 00007FF7AA83179F	0F297424 30	movaps xmmword ptr ss:[rsp+30],xmm6
• 00007FF7AA8317A4	48:8BDA	mov rbx,rdx
• 00007FF7AA8317A7	85C9	test ecx,ecx
• 00007FF7AA8317A9	✓ 74 77	je juskgosuc1ient_v4.7FF7AA831822
• 00007FF7AA8317AB	83F9 01	cmp ecx,1

결국 그 문제를 해결하기 위해서 위의 스크린샷처럼 text영역 중간에 넣어보려고도 했지만 공간이 너무 협소해서 실패했고 결론적으로는 어쩔 수 없이 그냥 pe bear프로그램을 깔아서 exe파일에 새로운 section cave를 추가해 권한을 부여하고 거기에 코드 케이를 작성하기로 결정했다.

00007FF5D4EC0000	0000000002000000	사용자	예약됨	PRV		-RW--
00007FF5D6EC0000	00000000000001000	사용자		PRV	-RW--	-RW--
00007FF5D6ED0000	00000000000001000	사용자		MAP	-R---	-R---
00007FF765ED0000	00000000000001000	사용자	cave.exe	IMG	-R---	ERWC-
00007FF765ED1000	0000000000001F000	사용자	".text"	IMG	ER---	ERWC-
00007FF765EF0000	00000000000011000	사용자	".rdata"	IMG	-R---	ERWC-
00007FF765F01000	00000000000001000	사용자	".data"	IMG	-RW--	ERWC-
00007FF765F02000	00000000000002000	사용자	".pdata"	IMG	-R---	ERWC-
00007FF765F04000	00000000000001000	사용자	".rsrc"	IMG	-R---	ERWC-
00007FF765F05000	00000000000001000	사용자	".reloc"	IMG	-R---	ERWC-
00007FF765F06000	00000000000001000	사용자	".cave"	IMG	ERWC-	ERWC-
00007FFAB2950000	00000000000001000	시스템	d3d9.dll	IMG	-R---	ERWC-

• 00007FF6ED286000	E8 FB28FEFF	call cave_v2.7FF6ED268900
• 00007FF6ED286005	90	nop
• 00007FF6ED286006	48:B8 0000000065CDCD	mov rax,41CDCD6500000000
• 00007FF6ED286010	6648:0F6EC0	movq xmm0,rax
• 00007FF6ED286015	90	nop
• 00007FF6ED286016	C3	ret
• 00007FF6ED286017	90	nop
• 00007FF6ED286018	0000	add byte ptr ds:[rax],a

이렇게 작성하고 원래 call 함수 대신에 이 부분으로 이동시키면 이제 영구 패치까진 됐다. 이제 그 다음으로는 이제 나 즉 여기서 TOMAS의 자산만 10억으로 만드는 것이 목표다.

35C0	test rax,rax	
52	je cave_v2.7FF6ED26B1B9	
3D0D EB5F0000	lea r9,qword ptr ds:[7FF6ED271159]	
3D05 C3660000	lea r8,qword ptr ds:[7FF6ED271838]	00007FF6ED271838:"username"
3D55 A8	lea rdx,qword ptr ss:[rbp-58]	
3BCC	mov rcx,r12	
2FD1FFFF	call cave_v2.7FF6ED2682B0	
	nop	
3 02	or ebx,2	
24 30	mov dword ptr ss:[rsp+30],ebx	

기본값 (x64)
 1: rcx 0000
 2: rdx 0000
 3: r8 0000
 4: r9 0000

00007FF6ED26B16E	4C:8D05 C3660000	lea r8,qword ptr ds:[7FF6ED271838]	
00007FF6ED26B175	48:8D55 A8	lea rdx,qword ptr ss:[rbp-58]	
00007FF6ED26B179	49:8BCC	mov rcx,r12	
00007FF6ED26B17C	E8 2FD1FFFF	call cave_v2.7FF6ED2682B0	
00007FF6ED26B181	90	nop	
00007FF6ED26B182	83CB 02	or ebx,2	
00007FF6ED26B185	895C24 30	mov dword ptr ss:[rsp+30],ebx	
00007FF6ED26B189	48:8BD0	mov rdx,rax	
00007FF6ED26B18C	48:8D4D 88	lea rcx,qword ptr ss:[rbp-78]	
00007FF6ED26B190	FF15 52500000	call qword ptr ds:[<public: static class QString	
00007FF6ED26B196	90	nop	
00007FF6ED26B197	83CB 04	or ebx,4	
00007FF6ED26B19A	895C24 30	mov dword ptr ss:[rsp+30],ebx	
00007FF6ED26B19E	45:33C0	xor r8d,r8d	
00007FF6ED26B1A1	48:8BD0	mov rdx,rax	
00007FF6ED26B1A4	48:8BCE	mov rcx,rsi	
00007FF6ED26B1A7	FF15 AB530000	call qword ptr ds:[<public: __cdecl QTableWidgetItem	
00007FF6ED26B1AD	48:8D05 AC8D0000	lea rax,qword ptr ds:[7FF6ED273F60]	

RAX 000001FB071E6220
 RBX 0000000000000000
 RCX 53D12AC7EB100000
 RDX 0000007B62EFB758 "tomas_3"
 RBP 0000007B62EFB7B0
 RSP 0000007B62EFB6B0
 RST 000001FB071F6220
 기본값 (x64 fastcall)
 1: rcx 53D12AC7EB100000 53D12AC7EB100000
 2: rdx 0000007B62EFB758 0000007B62EFB758 "tomas_3"
 3: r8 00007FF6ED271838 cave_v2.00007FF6ED271838 "username"
 4: r9 00007FF6ED271159 cave_v2.00007FF6ED271159
 5: [rsp+28] 000001DF000002F7 000001DF000002F7

그러기 위해서는 우선 내 이름에 해당하는 경우에 대해서만 자산을 바꿀 필요가 있기 때문에 우선 이름을 저장하는 레지스터의 위치를 먼저 찾아야 한다 랭킹에서는 이름도 보여주기 때문에 total근처에서 이름 관련 레지스터를 주석 덕분에 생각보다 쉽게 찾을 수 있었다. 이름은 rdx레지스터에서 저장되고 있었다.

0007FF6ED26B175	48:8D55 A8	lea rdx,qword ptr ss:[rbp-58]	
0007FF6ED26B179	49:8BCC	mov rcx,r12	
0007FF6ED26B17C	E8 2FD1FFFF	call cave_v2.7FF6ED2682B0	
0007FF6ED26B181	90	nop	
0007FF6ED26B182	83CB 02	or ebx,2	
0007FF6ED26B185	895C24 30	mov dword ptr ss:[rsp+30],ebx	
0007FF6ED26B189	48:8BD0	mov rdx,rax	
0007FF6ED26B18C	48:8D4D 88	lea rcx,qword ptr ss:[rbp-78]	
0007FF6ED26B190	FF15 52500000	call qword ptr ds:[<public: static class QString	
0007FF6ED26B196	90	nop	
0007FF6ED26B197	83CB 04	or ebx,4	

RAX 0000007B62EFB758 "tomas4"
 RBX 0000000000000002
 RCX 4A37F184FAE10000
 RDX 000001FB0A2B60C0 "tomas4"
 RBP 0000007B62EFB7B0
 RSP 0000007B62EFB6B0
 RST 000001FB0A237320

또한 위의 함수를 호출해서 반환값으로 이름을 받아 rdx레지스터에 저장하고 그 결과를 출력하고 있다는 점 또한 파악을 했다. ida에서 그 위치로 가보니

```

2      if ( v17 )
3      {
4          v18 = sub_7FF72D0882B0(v12, pExceptionObject, &qword_7FF72D091838, &unk_7FF72D091159);
5          v19 = v2 | 2;
6          LODWORD(v28) = v19;
7          v20 = (const struct QString *)QString::fromStdString(Src, v18);
8          v2 = v19 | 4;
9          LODWORD(v28) = v2;
10         QTableWidgetItem::QTableWidgetItem(v17, v20, 0);
11         *(_QWORD *)v17 = &off_7FF72D093F60;
12     }
13     else
14     {
15         v17 = nullptr;
16     }
17
18 0001A589 sub_7FF72D08AE70:137 (7FF72D08B189)

```

이렇게 v20에 저장된다는 걸 확인할 수 있었다. 여기서 어셈블리를 보면

```

.text:00007FF72D08B167      lea     r9, unk_7FF72D091159
.text:00007FF72D08B16E      lea     r8, qword_7FF72D091838
.text:00007FF72D08B175      lea     rdx, [rbp+40h+pExceptionObject]
.text:00007FF72D08B179      mov     rcx, r12
.text:00007FF72D08B17C      call    sub_7FF72D0882B0
.text:00007FF72D08B181      nop
.text:00007FF72D08B182      or      ebx, 2
.text:00007FF72D08B185      mov     dword ptr [rsp+140h+var_110], ebx
.text:00007FF72D08B189      mov     rdx, rax

```

이렇게 인자를 전달해 이름을 가져와서 그걸 v20에 저장한다는 걸 파악하고 이걸 만약 자산을 작성하는 함수에서 호출해서 이름 정보를 가져와서 내 이름이 맞는지 비교하기로 결정하고 그에 따라 케이브 코드를 아래와 같이 수정했다.

00007FF72D08B25C	49:8BCC	mov rcx, r12
• 00007FF72D08B25F	— E9 9CAD0100	jmp cave_v2.7FF72D0A6000
• 00007FF72D08B264	0F28C8	movaps xmm1, xmm0
• 00007FF72D08B267	45:33C9	xor r9d, r9d
• 00007FF72D08B26A	41:B0 66	mov r8b, 66
• 00007FF72D08B26D	48:8D4D 88	lea rcx, qword ptr ss:[rbp-78]
• 00007FF72D08B271	FF15 E14E0000	call qword ptr ds:[<public: static class QString

00007FF72D0A6000	E8 FB28FEFF	call cave_v2.7FF72D088900
00007FF72D0A6005	4C:8D0D 4DB1FEFF	lea r9, qword ptr ds:[7FF72D091159]
00007FF72D0A600C	4C:8D05 25B8FEFF	lea r8, qword ptr ds:[7FF72D091838]
00007FF72D0A6013	48:8D55 A8	lea rdx, qword ptr ss:[rbp-58]
00007FF72D0A6017	4C:89E1	mov rcx, r12
00007FF72D0A601A	E8 9122FEFF	call cave_v2.7FF72D0882B0
00007FF72D0A601F	83CB 02	or ebx, 2
00007FF72D0A6022	895C24 30	mov dword ptr ss:[rsp+30], ebx
00007FF72D0A6026	48:B8 0000000065CDCD	mov rax, 41CDCD6500000000
00007FF72D0A6030	6648:0F6EC0	movq xmm0, rax
00007FF72D0A6035	— E9 2A52FEFF	jmp cave_v2.7FF72D08B264
00007FF72D0A603A	0000	add byte ptr ds:[rax], al

우선 첫 번째는 이 함수를 스택 메모리 누수 없이 호출하기 위해 기존의 call에서 jmp로 코드 케이브 위치로 가는 방향으로 코드를 수정하고 우선 이름 호출하는 함수가 잘 작동하는지 확인하기 위해서 ida에서 본 함수 호출을 그대로 복사 붙여넣기해서 중단점을 걸어 레지스터값을 봐서 이름이 잘 찍히는지 확인해 본 결과

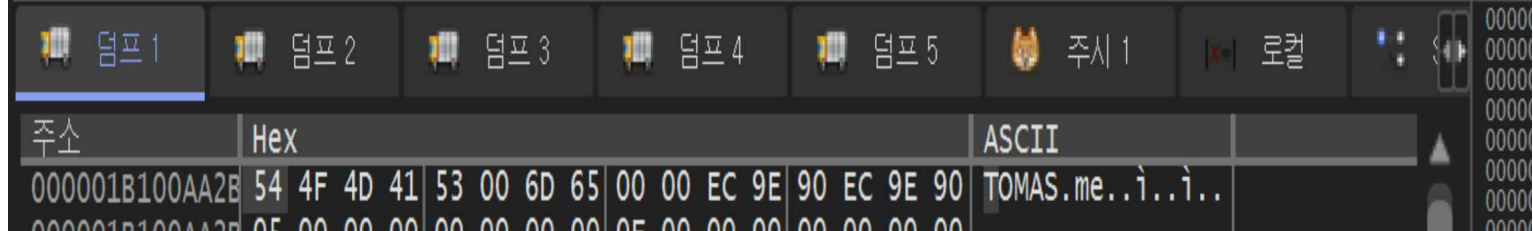
00007FF72D08B17C	E8 2FD1FFFF	call cave_v2.7FF72D0882B0	RAX	0000004350AFB6F8	"tomas_3"
00007FF72D08B181	90	nop	RBX	0000000000000002	
00007FF72D08B182	83CB 02	or ebx,2	RCX	6926D78AADA40000	
00007FF72D08B185	895C24 30	mov dword ptr ss:[rsp+30],ebx	RDX	000001B100B95C20	"tomas_3"
00007FF72D08B189	48:8BD0	mov rdx,rax	RBP	0000004350AFB750	
00007FF72D08B18C	48:8D4D 88	lea rcx,qword ptr ss:[rbp-78]	RSP	0000004350AFB650	
00007FF72D08B190	FF15 52500000	call qword ptr ds:[<public: static class QString	RST	000001R100AAAF310	
00007FF72D08B196	90	nop			
00007FF72D08B197	83CB 04	or ebx,4			
00007FF72D08B19A	895C24 30	mov dword ptr ss:[rsp+30],ebx			
00007FF72D08B19E	45:33C0	xor r8d,r8d			
00007FF72D08B1A1	48:8BD0	mov rdx,rax			
00007FF72D08B1A4	48:8BCE	mov rcx,rsi			
00007FF72D08B1A7	FF15 AB530000	call qword ptr ds:[<public: __cdecl QTableWidgetItem			
00007FF72D08B1AD	48:8D05 AC8D0000	lea rax,qword ptr ds:[7FF72D093F60]			

00007FF72D0A6000	E8 FB28FEFF	call cave_v2.7FF72D088900	RAX	000001B17D774418	
00007FF72D0A6005	4C:8D0D 4DB1FEFF	lea r9,qword ptr ds:[7FF72D091159]	RBX	0000000000000000	
00007FF72D0A600C	4C:8D05 25B8FEFF	lea r8,qword ptr ds:[7FF72D091838]	RCX	000001B100B931E0	
00007FF72D0A6013	48:8D55 A8	lea rdx,qword ptr ss:[rbp-58]	RDX	0000004350AFB6F8	"tomas_3"
00007FF72D0A6017	4C:89E1	mov rcx,r12	RBP	0000004350AFB750	
00007FF72D0A601A	E8 9122FEFF	call cave_v2.7FF72D0882B0	RSP	0000004350AFB650	
00007FF72D0A601F	83CB 02	or ebx,2	RST	000001R100AAAF380	
00007FF72D0A6022	895C24 30	mov dword ptr ss:[rsp+30],ebx			
00007FF72D0A6026	48:B8 0000000065CDCD	mov rax,41CDCD6500000000			
00007FF72D0A6030	6648:0F6EC0	movq xmm0,rax			
00007FF72D0A6035	E9 2A52FEFF	jmp cave_v2.7FF72D08B264			
00007FF72D0A603A	0000	add byte ptr ds:[rax],al			

위의 스크린샷처럼 코드 케이브에서 호출을 해도 인자만 잘 맞으면 잘 함수가 호출되고 올바른 값을 출력함을 확인했다. 이제는 여기서 이름이 TOMAS 즉 현재 로그인한 계정인 경우에만 자산을 10억으로 바꾸고 나머지는 다 0원으로 만들기 위해서는 현재 이름과 내 이름 TOMAS의 16진수 표현이 같은지만 확인해서 자산을 바꿔주면 된다. 그렇게 바꾸기 위해서 어셈블리 코드를

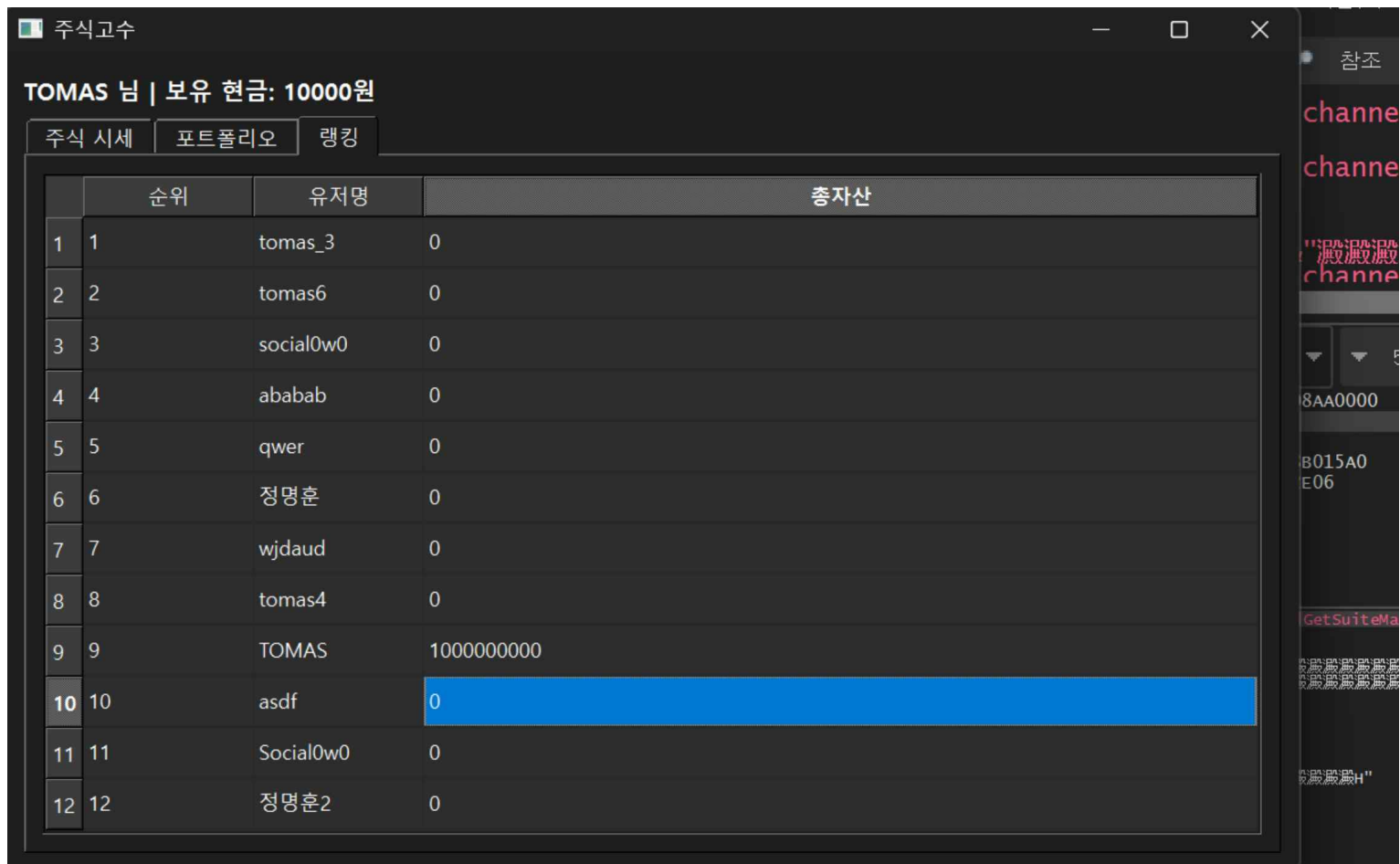
00007FF72D0A6022	895C24 30	mov dword ptr ss:[rsp+30],ebx
00007FF72D0A6026	49:BA 53414D4F540000	mov r10,544F4D4153
00007FF72D0A6030	4C:3912	cmp qword ptr ds:[rdx],r10
00007FF72D0A6033	74 07	je cave_v2.7FF72D0A603C
00007FF72D0A6035	E9 2A52FEFF	jmp cave_v2.7FF72D08B264
00007FF72D0A603A	0000	add byte ptr ds:[rax],al
00007FF72D0A603C	48:B8 0000000065CDCD	mov rax,41CDCD6500000000
00007FF72D0A6046	6648:0F6EC0	movq xmm0,rax
00007FF72D0A604B	E9 1452FEFF	jmp cave_v2.7FF72D08B264
00007FF72D0A6050	0000	add byte ptr ds:[rax],al
00007FF72D0A6052	0000	add byte ptr ds:[rax],al

위의 스크린샷에서 처럼 수정을 하고 실행을 시켰는데 이게 잘 안된다는 게 이상해 중단점을 걸고 확인해 보니



rdx레지스터가 가리키는 시작주소에서 값이 TOMAS + w0뒤에 null이 붙는데 나는 지금 cmp qword즉 8바이트 단위 비교를 하고 있어서 애초에 잘 될 수가 없었던 것이다.

그래서 cmp qword를 cmp dword즉 4바이트씩 두개씩 끊어서 비교하는 방식으로 코드를 다시 작성하니



이렇게 나만 10억을 가진 걸로 출력이 된다 물론 나는 실제 데이터를 바꾸진 않고 출력되는 결과만 바꾸었기 때문에 등수는 9등이긴 하지만 그래도 남들은 다 거지로 보이게 하고 나의 자산만 10억으로 바꾸는 데는 성공해 시스템의 무결성에 어느 정도 타격을 줬다.

한계는 진짜 데이터베이스 내부의 자산은 바꾸지 못했다는 점이다.나는 단지 출력되는 자산을 변경했을 뿐이다. 다음 주에 시간이 된다면 이번엔 보유 현금을 실제 내부 데이터를 바꿔 재벌이 되보려고 한다.

